INTERVIEW PREPARATION · C# AND .NET

C# for Python Developers

The core language, read through working code

This book teaches enough C# to read, change and discuss real code in an interview. Every chapter walks through a file in the companion project and shows the Python you would write for the same job. The last chapters are hands-on drills and a list of interview questions with short answers.

Study Guide · Generated 4 November 2026
Companion code: `~/repo/c.tutorial/` · .NET 10, C# 14

Contents

1 **What C# and .NET Are** 4

2 **The Companion Project** 7

3 **Syntax Basics** 10

4 **Types and Variables** 13

5 **Control Flow** 17

6 **Collections and Generics** 19

7 **Methods** 22

8 **Classes, Records and Structs** 27

9 **Inheritance and Interfaces** 33

10 **LINQ** 39

11 **Exceptions and Cleanup** 43

12 **Async and Await** 47

13 **Walkthrough: The Order System** 49

14 **Walkthrough: The Web API** 57

15 **Hands-on Drills** 60

16 **Interview Quick Reference** 63

A **Drill Solutions** 68

Audience and Scope

The reader writes Python and has not used C#. The goal is working fluency for an interview: read unfamiliar C#, make small changes, run the tests, and explain the core ideas in the right vocabulary. The goal is not to become a C# developer overnight.

Every C# listing labelled with a file path is copied from the companion project, which builds without warnings, and all 8 of its tests pass. Program output shown in the book is the real output of `dotnet run`.

If You Have Two Hours

- Read chapters 1, 4, 8, 9 and 13. They cover the topics of most interview questions.
- Skim chapter 10 (LINQ) and chapter 12 (async).
- Do drills 1 to 3 in chapter 15 at a keyboard.
- Read chapter 16 last, out loud.

1 What C# and .NET Are

C# is a statically typed, object-oriented language from Microsoft, first released in 2002. "Statically typed" means every variable's type is fixed and checked when the program is compiled, before it runs. .NET is the platform that compiles and runs C# code. People often say "C#" and ".NET" as if they were one thing; in an interview, use "C#" for the language and ".NET" for everything around it.

1.1 The Pieces

The table lists the five parts of .NET that interviewers name, with the nearest Python equivalent of each.

Piece	What it is	Python analogue
C#	The language: its syntax and rules.	Python the language
CLR	Common Language Runtime. The program that loads compiled code, runs it, and frees unused memory (garbage collection).	The CPython interpreter
Base Class Library	The built-in library: collections, files, HTTP, JSON, threads.	The standard library
.NET SDK	The `dotnet` command: create, build, run, test and package projects.	`python`, `pip`, `uv`, `pytest` together
NuGet	The public package registry, at nuget.org.	PyPI

1.2 How a C# Program Runs

C# is compiled in two steps, and the first step catches most mistakes. `dotnet build` runs the C# compiler, which checks every type in every file and refuses to produce output if any check fails. The output is a `.dll` file of IL (Intermediate Language), a CPU-independent instruction set. While the program runs, the CLR's JIT (just-in-time) compiler turns each method's IL into machine code for the current CPU the first time the method is called.

Python finds a type mistake only when the faulty line runs. C# finds it before the program can start. This is the largest practical difference between the two languages, and the reason C# code contains so many type names.

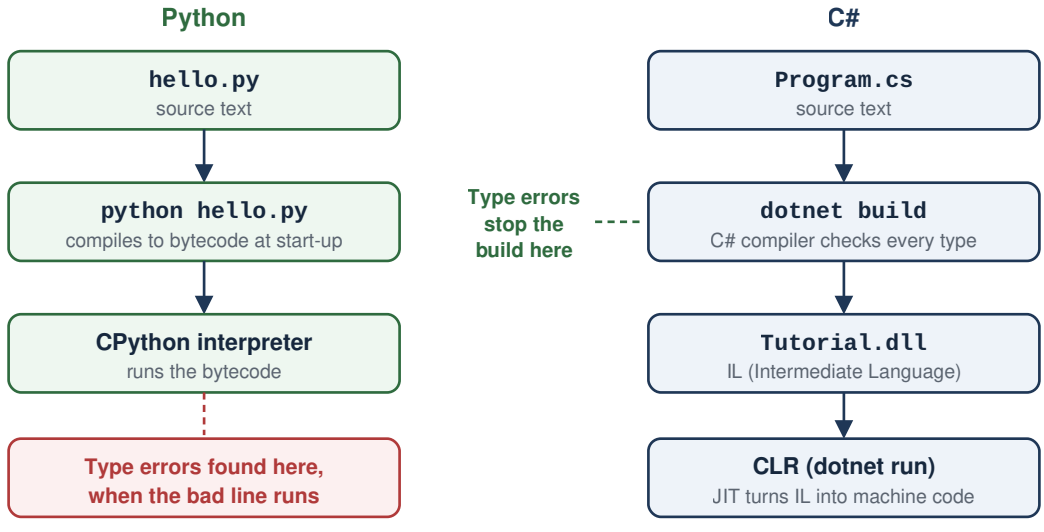


Figure 1. Python checks types while running; C# checks them before running.

1.3 .NET versus .NET Framework

Two products share the name, and job descriptions mix them up.

- **.NET Framework** (versions 1.0 to 4.8) is the original, Windows-only platform. It receives security fixes but no new features. Many company codebases still run on it.
- **.NET** (formerly .NET Core; versions 5, 6, 7, 8, 9, 10) is the cross-platform rewrite. It runs on Windows, Linux and macOS. All new work targets it. Even-numbered versions are LTS (long-term support) releases.

The companion project uses .NET 10, the LTS release from November 2025, with C# 14. A codebase "on Framework" uses older syntax, such as explicit `Main` methods and namespace blocks in braces. Section 3.2 shows both styles.

1.4 Python and C# Side by Side

Topic	Python	C#
Typing	Dynamic; type hints are optional and not enforced	Static; every type checked at compile time
Blocks	Indentation	`{ }` braces; statements end in `;`

Topic	Python	C#
Code location	Functions may live anywhere in a module	Every method lives inside a class
Inheritance	Multiple base classes allowed	One base class, any number of interfaces
Privacy	`_name` by convention only	`private`, enforced by the compiler
Threads	Default CPython build has the GIL: one thread runs Python at a time	Threads run truly in parallel
Memory	Reference counting plus a cycle collector	Tracing garbage collector
Speed	Interpreted; slower for CPU work	JIT-compiled; usually much faster

2 The Companion Project

The companion project at `~/repo/c.tutorial` is a .NET solution with three projects. A **solution** (`.slnx` file) is a list of projects that build together. A **project** (`.csproj` file) compiles to one `.dll` or executable. A Python monorepo with several packages, each with its own `pyproject.toml`, is the closest match.

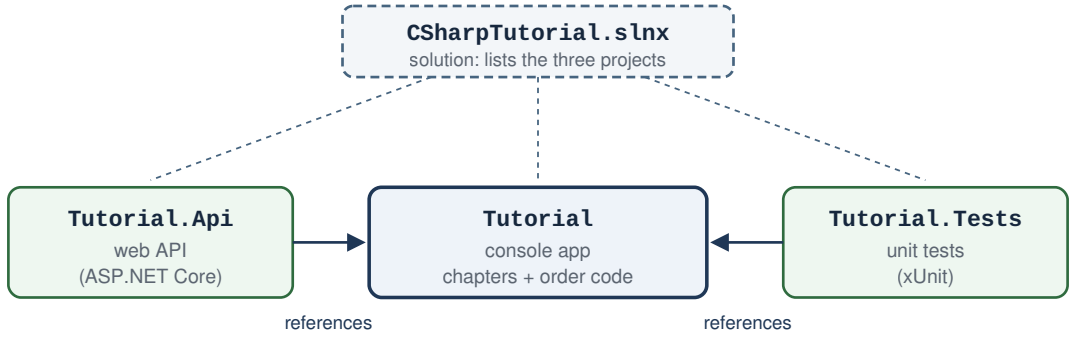


Figure 2. The solution. The API and the tests both use the order code inside `Tutorial`.

2.1 Files

Path	Contents
`src/Tutorial/Program.cs`	Entry point. Runs one chapter or all of them.
`src/Tutorial/Ch03Basics.cs` … `Ch12Async.cs`	One file per book chapter. The number in the file name is the chapter number.
`src/Tutorial/Orders/`	The order system: products, orders, discounts, checkout.
`src/Tutorial/Ch13Orders.cs`	Runs the order system end to end.
`src/Tutorial.Api/Program.cs`	A web API that sells products through the order system.
`tests/Tutorial.Tests/`	Unit tests for `BankAccount` and `OrderService`.

2.2 The dotnet Command

One command, `dotnet`, replaces the separate Python tools. Run these from the repo root. The SDK is installed at `~/.dotnet`; if `dotnet` is not found, run `export PATH=$HOME/.dotnet:$PATH` first.

Task	Python	.NET
Run all chapters	`python main.py`	`dotnet run --project src/Tutorial`
Run chapter 6	`python main.py 6`	`dotnet run --project src/Tutorial -- 6`
Compile only	(none)	`dotnet build`
Run tests	`pytest`	`dotnet test`
Add a package	`uv add requests`	`dotnet add package Serilog`
New project	`uv init`	`dotnet new console -o MyApp`
Format code	`ruff format`	`dotnet format`
Virtual env	`.venv`	None needed. Packages are per project.

The demo numbers match this book's chapter numbers: `-- 6` runs the code for chapter 6, and the valid numbers are 3 to 13. The `--` separates `dotnet`'s own options from the arguments passed to your program.

2.3 The Project File

The `.csproj` file is an XML file that lists the project's build settings and package dependencies. It is the counterpart of `pyproject.toml`.

Tutorial/Tutorial.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <RootNamespace>Tutorial</RootNamespace>
  </PropertyGroup>
</Project>
```

- `OutputType Exe`: build a runnable program, not a library.
- `TargetFramework net10.0`: the .NET version (Python: `requires-python`).
- `ImplicitUsings`: imports common namespaces such as `System` and `System.Linq` into every file automatically.
- `Nullable`: makes the compiler warn when code might use a `null` value (section 4.5).

A test project adds `<PackageReference>` lines for xUnit, the same way `pyproject.toml` lists dependencies.

2.4 Program.cs Walkthrough

`Program.cs` reads the command line and calls each chapter's `Run` method. It uses syntax from later chapters (`foreach`, `switch`, the `? :` conditional, LINQ's `Select`); skim it now and return to it after chapter 10. It uses **top-level statements**: code written directly in the file, with no class around it. The compiler wraps it in a hidden class and `Main` method. Only one file per project may do this.

Tutorial/Program.cs (abridged)

```
using Tutorial;

var titles = new Dictionary<int, string>
{
    [1] = "Hello, C#",
    [2] = "Types and variables",
    // ... chapters 3 to 11 ...
};

// args is the command-line arguments, like Python's sys.argv[1:]
IEnumerable<int> toRun = args.Length > 0 ? args.Select(int.Parse) : titles.Keys;

foreach (int chapter in toRun)
{
    Console.WriteLine($"\n=== Chapter {chapter}: {titles[chapter]} ===");
    switch (chapter)
    {
        case 1: Ch03Basics.Run(); break;
        // ...
        case 10: await Ch12Async.RunAsync(); break;
        case 11: Ch13Orders.Run(); break;
    }
}
```

- `using Tutorial;` imports the `Tutorial` namespace, where the chapter classes live. It is Python's `from tutorial import *`, limited to type names.
- `args.Select(int.Parse)` converts each argument string to an `int` (Python: `map(int, sys.argv[1:])`). Chapter 10 covers `Select`.
- `titles.Keys` runs every chapter when no argument is given.
- C# does not let one `case` fall through into the next, so each one ends in `break` (or `return` or `throw`).

3 Syntax Basics

C# uses C-style syntax. Braces mark blocks, semicolons end statements, and names are case-sensitive. Indentation is ignored by the compiler but kept by convention. Run this chapter with `dotnet run --project src/Tutorial -- 3`.

3.1 A First Program

Tutorial/Ch03Basics.cs

```
namespace Tutorial;

// Chapter 3: the shape of a C# file.
public static class Ch03Basics
{
    public static void Run()
    {
        Console.WriteLine("Hello, C#!");

        string name = "Ada";
        int year = 1843;

        // $"..." is an interpolated string, like Python's f"..."
        Console.WriteLine($"{name} wrote the first program in {year}.");

        // Format specifiers go after a colon. F2 = two decimal places.
        // Python: f"{pi:.2f}"
        double pi = Math.PI;
        Console.WriteLine($"pi to 2 places: {pi:F2}");

        // Statements end with ';'. Blocks use { } instead of indentation.
        if (year < 1900)
        {
            Console.WriteLine("That was a long time ago.");
        }
    }
}
```

The same program in Python:

Python

```python
import math

def run():
    print("Hello, C#!")
    name = "Ada"
    year = 1843
    print(f"{name} wrote the first program in {year}.")
    print(f"pi to 2 places: {math.pi:.2f}")
    if year < 1900:
        print("That was a long time ago.")
```

- `namespace Tutorial;` puts everything in the file into the `Tutorial` namespace. A namespace is a dotted name, such as `Tutorial.Orders`, that becomes part of the full name of every type declared in it, like a Python package path.
- `public static class` is a class that holds only static methods and fields and cannot be instantiated. It is the closest thing C# has to a Python module of plain functions.
- `public static void Run()` declares a method. `void` means it returns nothing (Python: `-> None`).
- `string name = "Ada";` declares the variable's type before its name.
- `$"...{name}..."` is an interpolated string, the C# version of an f-string.
- Strings use double quotes only. Single quotes hold one character: `'A'` is a `char`.

3.2 The Older Style

Code written before 2021, and all .NET Framework code, spells out a namespace block, a class and a `Main` method, all of which modern C# lets you leave out. The program below is the same "hello" in that style.

C#

```
using System;

namespace Tutorial
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, C#!");
        }
    }
}
```

`static void Main(string[] args)` is the entry point: the method the runtime calls first. It plays the role of Python's `if __name__ == "__main__":` block. The namespace wraps the class in braces instead of ending in a semicolon. Both styles compile to the same program.

3.3 Naming Conventions

C# naming differs from PEP 8, and interviewers notice. Methods and properties use PascalCase, where Python uses snake_case.

Kind of name	C# style	Python style
Class, record, struct	`BankAccount`	`BankAccount`
Method	`Deposit(), TryParse()`	`deposit(), try_parse()`
Property, public field	`Balance, Owner`	`balance, owner`
Local variable, parameter	`openingBalance`	`opening_balance`
Private field	`_balance`	`_balance`
Interface	`IShape (I prefix)`	`Shape (Protocol)`
Constant	`TaxRate`	`TAX_RATE`
Async method	`FetchAsync()`	`fetch()`

4 Types and Variables

Every C# variable has one type for its whole life, and the compiler rejects any assignment that does not fit. In Python, `x = 3` followed by `x = "three"` is legal. In C#, `int x = 3;` followed by `x = "three";` does not compile. File: `src/Tutorial/Ch04Types.cs`.

4.1 Built-in Types

C# type	Holds	Python closest
`int`	Whole number, 32-bit, about ±2.1 billion. Wraps around silently on overflow.	`int (unbounded)`
`long`	Whole number, 64-bit. Literal suffix `L`.	`int`
`double`	64-bit binary floating point.	`float`
`decimal`	128-bit base-10 number. Exact for money. Literal suffix `m`.	`decimal.Decimal`
`bool`	`true` or `false`, lower case.	`bool`
`char`	One character, in single quotes.	(a 1-char str)
`string`	Immutable text, in double quotes.	`str`
`object`	Any value at all. Every type derives from it.	`object`

Tutorial/Ch04Types.cs

```
// in Run():
int count = 3;                // 32-bit whole number
long big = 9_000_000_000L;    // 64-bit whole number
double ratio = 0.1 + 0.2;     // binary floating point, like Python float
decimal price = 0.1m + 0.2m; // base-10 number, exact for money
bool done = false;            // lowercase true/false
char initial = 'A';           // single quotes = one character
string greeting = "hi";       // double quotes = string

Console.WriteLine($"double: {ratio}   decimal: {price}");
Console.WriteLine($"{count} {big} {done} {initial} {greeting}");
```

Output

```
double: 0.30000000000000004   decimal: 0.3
```

Use `decimal` for money, as the order system in chapter 13 does. A `double` cannot store 0.1 exactly, so sums of prices drift by tiny amounts.

4.2 var

`var` tells the compiler to work out the type from the right-hand side. The variable is still statically typed. `var` saves typing; it does not make the variable dynamic.

Tutorial/Ch04Types.cs

```
// in Run():
// var asks the compiler to infer the type. The type is still fixed.
var total = count * 2;       // total is an int
// total = "six";            // compile error: cannot convert string to int
Console.WriteLine($"total is an {total.GetType().Name}");
```

Output

```
total is an Int32
```

`Int32` is the .NET name for `int`; `int` is the C# keyword for it. Most teams use `var` when the type is obvious from the line, such as `var names = new List<string>();`.

4.3 Conversions

C# converts between number types automatically only when the conversion is considered safe, such as `int` to `long` or `int` to `double`. A conversion that can lose information, such as `double` to `int`, must be written out.

Tutorial/Ch04Types.cs

```
// in Run():
// Converting between types
int parsed = int.Parse("42");                  // throws on bad input
bool ok = int.TryParse("abc", out int maybe);  // returns false instead
double widened = count;                        // implicit: no data lost
int narrowed = (int)3.99;                      // explicit cast: 3
Console.WriteLine($"{parsed} {ok} {maybe} {widened} {narrowed}");
```

Output

```
42 False 0 3 3
```

- `int.Parse("42")` is Python's `int("42")`. It throws an exception on bad input.
- `int.TryParse(text, out int maybe)` returns `true` or `false` instead of throwing, and writes the number into `maybe`. This `TryX` pattern appears all over .NET. Section 7.3 explains `out`.
- `(int)3.99` is a **cast**: an explicit conversion. It truncates to 3.
- `7 / 2` is 3, because both sides are `int`. Python's `/` would give 3.5.

4.4 Value Types and Reference Types

Every C# type is either a value type or a reference type, and the two behave differently on assignment. This is the single most common C# interview question.

- A **value type** variable holds the data itself. Assigning it copies the data. Value types: all number types, `bool`, `char`, `struct`, `enum`.
- A **reference type** variable holds the address of an object. Assigning it copies the address, so both variables point at one object. Reference types: `class`, `record`, `string`, arrays, `List`, interfaces.

Python has only the second kind: every Python variable is a reference.

Tutorial/Ch04Types.cs

```
// in Run():
// Value types (int, double, struct) are copied on assignment.
int a = 1;
int b = a;
b = 99;
Console.WriteLine($"a = {a}, b = {b}");         // a is still 1

// Reference types (classes, List, arrays) share one object.
var listA = new List<int> { 1, 2 };
var listB = listA;
listB.Add(3);
Console.WriteLine($"listA.Count = {listA.Count}");  // 3
```

Output

```
a = 1, b = 99
listA.Count = 3
```



Figure 3. Assigning a value type copies the data; assigning a reference type copies only the address, so both variables point at one object.

4.5 null

`null` means "no object", like Python's `None`. A type name followed by `?` marks a variable that is allowed to hold `null`.

Tutorial/Ch04Types.cs

```
// in Run():
// null and nullable types
int? missing = null;                    // int? = int that may be null
Console.WriteLine(missing ?? -1);       // ?? = "if null, use this"
string? maybeName = null;               // string? = may be null
Console.WriteLine(maybeName?.Length ?? 0); // ?. = stop if null
```

Output

```
-1
0
```

- `int?` is an `int` that may also be `null` (Python: `Optional[int]`).
- `int?` and `string?` look alike but work differently. `int?` is a separate type, `Nullable<int>`, with `.HasValue` and `.Value`. `string?` is only a note to the compiler: at run time any `string` variable can hold `null`, and the `?` controls which warnings you get.
- `x ?? y` gives `x`, or `y` if `x` is null. Python's `x if x is not None else y`.
- `x?.Length` gives `null` instead of crashing when `x` is null.
- With `<Nullable>enable</Nullable>` in the project file, the compiler warns when a `string` without `?` might receive `null`. These are **nullable reference types**. They are warnings, not errors.
- Using a null reference throws `NullReferenceException`, the C# version of `AttributeError: 'NoneType' object has no attribute ...`.

5 Control Flow

Branches and loops work as in Python, written with parentheses and braces. File: `src/Tutorial/Ch05ControlFlow.cs`.

5.1 Branches

Tutorial/Ch05ControlFlow.cs

```
// in Run():
int score = 83;

if (score >= 90)
{
    Console.WriteLine("A");
}
else if (score >= 80)   // Python: elif
{
    Console.WriteLine("B");
}
else
{
    Console.WriteLine("C or below");
}

// && || !  instead of  and or not
bool passed = score >= 60 && score <= 100;

// Ternary: condition ? a : b      (Python: a if condition else b)
string label = passed ? "pass" : "fail";
Console.WriteLine(label);

// switch expression: picks a value by pattern (like Python's match)
string grade = score switch
{
    >= 90 => "A",
    >= 80 => "B",
    >= 70 => "C",
    _ => "F",           // _ = anything else
};
Console.WriteLine($"grade {grade}");
```

- The condition of an `if` must be in parentheses and must be a `bool`. C# has no "truthiness": `if (count)` does not compile. Write `if (count > 0)`.
- `&&`, `||` and `!` are Python's `and`, `or` and `not`.
- `cond ? a : b` is Python's `a if cond else b`.
- A **switch expression** picks a value by testing the input against patterns in order. `_` matches anything. It resembles Python's `match` statement, but it returns a value.

5.2 Loops

Tutorial/Ch05ControlFlow.cs

```
// in Run():
// Classic for loop: start; keep going while; step
// Python: for i in range(3):
for (int i = 0; i < 3; i++)
{
    Console.Write($"{i} ");
}
Console.WriteLine();

// foreach is Python's for ... in ...
string[] fruits = { "apple", "banana", "cherry" };
foreach (string fruit in fruits)
{
    if (fruit == "banana")
    {
        continue;
    }
    Console.WriteLine(fruit);
}

int n = 0;
while (n < 3)
{
    n++;                // same as n += 1
}
Console.WriteLine($"n = {n}");
```

Python	C#
`for i in range(3):`	`for (int i = 0; i < 3; i++)`
`for fruit in fruits:`	`foreach (string fruit in fruits)`
`for i, f in enumerate(fruits):`	`for (int i = 0; i < fruits.Length; i++) then fruits[i]`
`while n < 3:`	`while (n < 3)`
`n += 1`	`n++; or n += 1;`

The `for` header has three parts separated by semicolons: the starting statement, the condition checked before each pass, and the step run after each pass. `break` and `continue` work as in Python.

6 Collections and Generics

Every C# collection holds one declared element type. A `List<string>` holds only strings, and adding an `int` is a compile error. File: `src/Tutorial/Ch06Collections.cs`.

6.1 Python to C# Mapping

Python	C#	Notes
`list`	`List<T>`	Growable. `.Add`, `.Remove`, `.Count`.
`list (fixed)`	`T[]`	Array. Fixed length. `.Length`.
`dict`	`Dictionary<K, V>`	Missing key throws `KeyNotFoundException`.
`set`	`HashSet<T>`	`.Add` returns false if already present.
`tuple`	`(string, int)`	Parts can be named: `(string City, int Pop)`.
`collections.deque`	`Queue<T>, Stack<T>`	FIFO and LIFO.
`Iterable`	`IEnumerable<T>`	Anything `foreach` can loop over.

The count is `.Count` on lists and dictionaries, but `.Length` on arrays and strings. There is no single `len()`.

6.2 Walkthrough

Tutorial/Ch06Collections.cs

```
// in Run():
// Array: fixed length, one element type
int[] scores = { 90, 72, 85 };
Console.WriteLine($"first {scores[0]}, length {scores.Length}");
Console.WriteLine($"last {scores[^1]}");        // Python: scores[-1]
int[] firstTwo = scores[0..2];                  // Python: scores[0:2]
Console.WriteLine(string.Join(", ", firstTwo));

// List<T>: growable, like a Python list, but one element type
var names = new List<string> { "Ada", "Grace" };
names.Add("Linus");                             // Python: append
names.Remove("Grace");
Console.WriteLine($"{names.Count} names: {string.Join(", ", names)}");
```

Indexing from the end uses `^1`, not `-1`. Slicing uses `..` in place of `:`. A negative index throws `IndexOutOfRangeException`.

Tutorial/Ch06Collections.cs

```
// in Run():
// Dictionary<TKey, TValue>: like a Python dict
var ages = new Dictionary<string, int>
{
    ["Ada"] = 36,
    ["Grace"] = 85,
};
ages["Linus"] = 54;                             // add or overwrite

// ages["Nobody"] would throw KeyNotFoundException (Python: KeyError)
if (ages.TryGetValue("Ada", out int adaAge))    // Python: ages.get("Ada")
{
    Console.WriteLine($"Ada was {adaAge}");
}
foreach (var (person, age) in ages)             // Python: ages.items()
{
    Console.WriteLine($"{person}: {age}");
}
```

- The braces after `new Dictionary<string, int>` are a **collection initializer**: starting contents, like a dict literal.
- `TryGetValue` is the safe lookup. It returns `false` when the key is absent, where Python's `.get()` returns `None`.
- `foreach (var (person, age) in ages)` unpacks each key-value pair, like `for person, age in ages.items()`.

6.3 Generics

A **generic type** is a type with a placeholder for another type, written in angle brackets. `List<T>` is written once; `List<string>` and `List<int>` are two concrete types made from it. Python's `list[str]` hint looks the same but is never enforced. C# enforces it at compile time, and the type argument still exists at run time.

You can write your own generic methods and classes. `<T>` after the method name declares the placeholder, and the caller's argument types fill it in:

C#

```
public static T FirstItem<T>(List<T> items)
{
    return items[0];
}

string name = FirstItem(new List<string> { "Ada", "Grace" });   // T is string
int score = FirstItem(new List<int> { 90, 72 });                // T is int
```

A `where` clause limits what `T` may be. For example, `where T : IShape` accepts only types that implement the `IShape` interface (chapter 9).

7 Methods

A method is a function that belongs to a class. C# has no free-standing functions; even utility functions live in a `static class`. File: `src/Tutorial/Ch07Methods.cs`.

7.1 Anatomy of a Signature

Tutorial/Ch07Methods.cs

```
// access  static  return-type  Name(type param, type param)
public static int Add(int a, int b)
{
    return a + b;
}
```

Read the first line left to right:

- `public`: who may call it (section 7.2).
- `static`: it belongs to the class, not to an object. Call it as `Ch07Methods.Add(2, 3)`.
- `int`: the return type. Every return path must return an `int`.
- `Add`: the name, in PascalCase.
- `(int a, int b)`: each parameter with its type.

Python equivalent: `def add(a: int, b: int) -> int:`, except C# enforces the types.

A method with a single expression can use `=>` instead of braces and `return`:

Tutorial/Ch07Methods.cs

```
// Expression-bodied method: one expression, no braces, no return
public static int Square(int x) => x * x;
```

Overloading means two methods share a name and differ in parameter types. The compiler picks one from the argument types. Python has no overloading; it would use one function that checks types.

Tutorial/Ch07Methods.cs

```
// Overload: same name, different parameter types
public static double Add(double a, double b) => a + b;
```

7.2 Access Modifiers and static

An access modifier states which code may use a class member. The compiler enforces it.

Modifier	Who can use the member
`public`	Any code.
`private`	Only code inside the same class. The default for class members.
`protected`	The same class and its subclasses.
`internal`	Any code in the same project (assembly). The default for classes.

`static` is not an access modifier. A `static` member belongs to the class itself: one copy shared by all objects, used without creating an object. A non-static (instance) member belongs to each object. In Python terms, `static` methods are `@staticmethod`, and static fields are class attributes.

7.3 Parameters

Tutorial/Ch07Methods.cs

```
// Optional parameter with a default, as in Python
public static string Greet(string name, string greeting = "Hello")
    => $"{greeting}, {name}!";

// out parameter: hand back a second value
public static bool TryDivide(int a, int b, out int result)
{
    if (b == 0)
    {
        result = 0;
        return false;
    }
    result = a / b;
    return true;
}

// Or return a tuple, as Python would
public static (int Min, int Max) MinMax(List<int> values)
    => (values.Min(), values.Max());

// params: any number of arguments (Python: *args)
public static int Sum(params int[] numbers)
{
    int total = 0;
    foreach (int n in numbers)
    {
        total += n;
    }
    return total;
}

// ref: the method can change the caller's variable
public static void Double(ref int x) => x *= 2;
```

The calls, from `Run`:

Tutorial/Ch07Methods.cs

```
// in Run():
Console.WriteLine(Greet("Ada"));
Console.WriteLine(Greet("Ada", greeting: "Welcome"));   // named argument

if (TryDivide(10, 3, out int quotient))
{
    Console.WriteLine($"10 / 3 = {quotient}");
}

var (lo, hi) = MinMax(new List<int> { 4, 1, 9 });       // unpack a tuple
Console.WriteLine($"min {lo}, max {hi}");
Console.WriteLine(Sum(1, 2, 3, 4));

int value = 5;
Double(ref value);
Console.WriteLine($"value is now {value}");
```

- **Default values** work as in Python: `string greeting = "Hello"`.
- **Named arguments** use a colon: `Greet("Ada", greeting: "Welcome")`.
- **`out`** marks a parameter the method must fill in. The caller writes `out int quotient` and gets the value back. It exists because C# methods historically returned only one value.
- **Tuples** are the modern way to return several values, as in Python. `var (lo, hi) = MinMax(...)` unpacks them.
- **`params`** accepts any number of arguments as an array, like `*args`.
- **`ref`** passes the caller's variable itself, so the method can change it. Python has no equivalent.

7.4 Lambdas, Func and Action

A lambda is an unnamed function written `parameters => expression`.
Python writes `lambda x: x * 3`; C# writes `x => x * 3`. A C# lambda may also have a braced body with several statements, which Python lambdas cannot.

Tutorial/Ch07Methods.cs

```
// in Run():
// Lambdas. Func<int, int>: takes an int, returns an int.
// Action<string>: takes a string, returns nothing.
Func<int, int> triple = x => x * 3;     // Python: lambda x: x * 3
Action<string> shout = s => Console.WriteLine(s.ToUpper());
Console.WriteLine(triple(5));
shout("done");
```

A variable that holds a function has a **delegate** type: a type that states the function's parameter types and return type. `Func<int, int>` takes an `int` and returns an `int`; the last type argument is always the return type. `Action<string>` takes a `string` and returns nothing.

8 Classes, Records and Structs

C# offers three ways to define your own type. A `class` is the default. A `record` is for data compared by value. A `struct` is a small value type. File: `src/Tutorial/Ch08Classes.cs`.

8.1 A Class: BankAccount

`BankAccount` shows **encapsulation**, which means a class keeps its data private and lets outside code change it only through methods that check each change. Outside code can read the balance but can change it only through `Deposit` and `Withdraw`.

Tutorial/Ch08Classes.cs

```
// A class bundles data with the methods that act on it.
public class BankAccount
{
    // Static field: shared by all BankAccounts (Python: class attribute)
    private static int _nextId = 1;

    // Instance field: private storage, one per object
    private decimal _balance;

    // Read-only auto-properties: set in the constructor, never again
    public string Id { get; }
    public string Owner { get; }

    // Computed property (Python: @property)
    public decimal Balance => _balance;

    // Constructor (Python: __init__). No "self": fields are in scope directly.
    public BankAccount(string owner, decimal openingBalance = 0m)
    {
        if (string.IsNullOrWhiteSpace(owner))
        {
            throw new ArgumentException("Owner is required.", nameof(owner));
        }
        Owner = owner;
        Id = $"ACC-{_nextId++:D4}";
        _balance = openingBalance;
    }

    public void Deposit(decimal amount)
    {
        if (amount <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount), "Deposit must be positive.");
        }
        _balance += amount;
    }

    public void Withdraw(decimal amount)
    {
        if (amount > _balance)
        {
            throw new InvalidOperationException(
                $"Insufficient funds: balance {_balance}, requested {amount}.");
        }
        _balance -= amount;
    }

    // Python: __str__
    public override string ToString() => $"{Id} {Owner}: {Balance:0.00}";
}
```

The same class in Python, for comparison:

Python

```python
class BankAccount:
    _next_id = 1                        # class attribute

    def __init__(self, owner: str, opening_balance=Decimal("0")):
        if not owner or not owner.strip():
            raise ValueError("Owner is required.")
        self.owner = owner
        self.id = f"ACC-{BankAccount._next_id:04d}"
        BankAccount._next_id += 1
        self._balance = opening_balance

    @property
    def balance(self) -> Decimal:
        return self._balance

    def deposit(self, amount: Decimal) -> None:
        if amount <= 0:
            raise ValueError("Deposit must be positive.")
        self._balance += amount

    def __str__(self) -> str:
        return f"{self.id} {self.owner}: {self.balance:.2f}"
```

8.2 Reading the Class

- **Field**: a variable stored in each object. `private decimal _balance;` Fields are normally private.
- **Property**: a member that looks like a field from outside but runs code on read or write. `public string Owner { get; }` can be read by anyone and set only in the constructor. `public decimal Balance => _balance;` is computed on each read, like `@property`. Public data in C# is exposed through properties, not public fields.
- **Constructor**: the method with the class's name and no return type. It is `__init__`. There is no `self` parameter; members are in scope directly, and `this` refers to the current object when you need it.
- **`nameof(owner)`** produces the string `"owner"`. If the parameter is renamed and `nameof` is not, the build fails, so the text cannot go out of date.
- **`override string ToString()`** replaces the default text form, as `__str__` does. `Console.WriteLine(account)` calls it.

Property forms you will meet in real code:

C#

```
public string Name { get; set; }          // anyone reads and writes
public string Name { get; private set; }  // anyone reads; only this class writes
public string Name { get; }               // set only in the constructor
public string Name { get; init; }         // set in constructor or object
initializer
public string Name => _name;              // computed on every read
```

8.3 Using It

Tutorial/Ch08Classes.cs

```
// in Run():
var account = new BankAccount("Ada", 100m);   // "new" creates an object
account.Deposit(50m);
account.Withdraw(30m);
Console.WriteLine(account);                   // calls ToString()
// account.Balance = 1_000_000m;              // compile error: no setter

try
{
    account.Withdraw(500m);
}
catch (InvalidOperationException ex)
{
    Console.WriteLine($"Refused: {ex.Message}");
}
```

Output

```
ACC-0001 Ada: 120.00
Refused: Insufficient funds: balance 120, requested 500.
```

`new BankAccount(...)` creates an object; Python omits the word `new`. The commented-out line assigns to `Balance`. It fails to compile because the property has no setter.

8.4 Records

A **record** is a class designed to hold data, with value equality: two records are equal when all their properties are equal. One line declares the properties, the constructor, equality and a readable `ToString`. It matches Python's `@dataclass(frozen=True)`.

Tutorial/Ch08Classes.cs

```
// record: a data class with value equality (Python: @dataclass(frozen=True))
public record Point(int X, int Y);

// in Run():
var p1 = new Point(1, 2);
var p2 = new Point(1, 2);
Console.WriteLine(p1 == p2);          // True: records compare by value
Console.WriteLine(p1);                // Point { X = 1, Y = 2 }
var moved = p1 with { X = 10 };       // copy with one field changed
Console.WriteLine(moved);

var g1 = new BankAccount("Grace");
var g2 = new BankAccount("Grace");
Console.WriteLine(g1 == g2);          // False: classes compare by identity
```

Output

```
True
Point { X = 1, Y = 2 }
Point { X = 10, Y = 2 }
False
```

`p1 == p2` is `True` because records compare contents. `g1 == g2` is `False` because classes compare identity: two separate objects are never equal unless the class overrides equality. Python's `==` on plain classes also compares identity.

`with` makes a copy with some properties changed, like `dataclasses.replace`.

8.5 Structs

A **struct** is a value type you define yourself (section 4.4). Assigning one copies all of its data. Structs suit small values such as coordinates or temperatures.

`new Temperature { Celsius = 20 }` is an **object initializer**: it creates the object, then sets each listed property. It works for classes too.

Tutorial/Ch08Classes.cs

```
// struct: a small value type, copied on assignment
public struct Temperature
{
    public double Celsius { get; set; }
    public double Fahrenheit => Celsius * 9 / 5 + 32;
}

// in Run():
var t1 = new Temperature { Celsius = 20 };
var t2 = t1;                          // struct: a full copy
t2.Celsius = 100;
Console.WriteLine($"t1 {t1.Celsius}C = {t1.Fahrenheit}F; t2 {t2.Celsius}C");
```

Output

```
t1 20C = 68F; t2 100C
```

8.6 Choosing Between Them

	class	record	struct
Kind	Reference	Reference	Value
== compares	Identity	Contents	Not defined unless you write it; `Equals` compares contents
Typical use	Objects with behaviour and changing state	Data passed around: DTOs, results, messages	Small values, a few fields
Example here	`BankAccount`, `Order`	`Product`, `Receipt`	`Temperature`

A **DTO** (data transfer object) is a type that only holds data moving between parts of a program, such as a web request body.

9 Inheritance and Interfaces

A class may inherit from one base class and implement any number of interfaces. An **interface** is a named list of method and property signatures that a class must implement; it holds no stored data. Code that accepts the interface works with every class that implements it. File: `src/Tutorial/Ch09Interfaces.cs`.

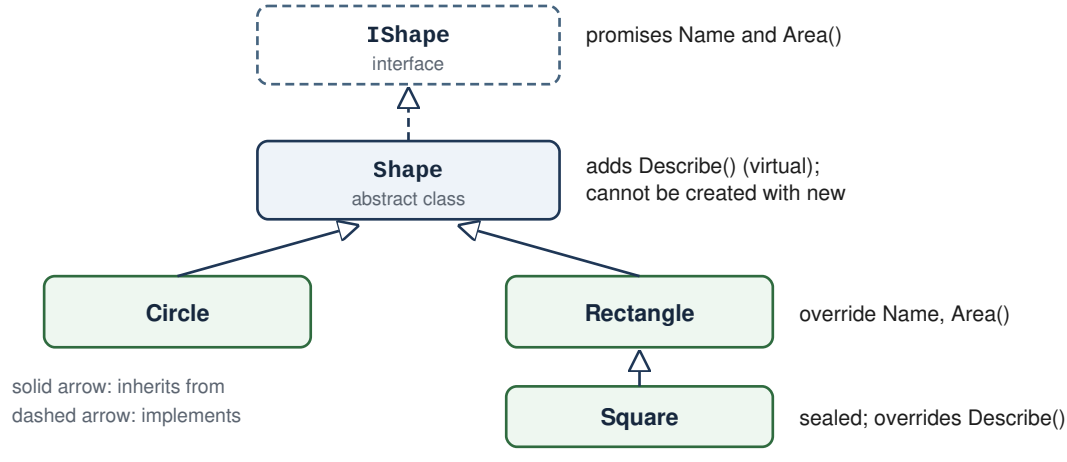


Figure 4. The shape types in Ch09Interfaces.cs.

9.1 The Interface and the Base Class

Tutorial/Ch09Interfaces.cs

```
// Interface: a list of members a class promises to provide.
// Normally it contains no code.
// Python equivalent: typing.Protocol or an ABC with only abstract methods.
public interface IShape
{
    string Name { get; }
    double Area();
}

// Abstract class: cannot be created directly. Shares code with subclasses
// and leaves some members for them to fill in.
public abstract class Shape : IShape
{
    public abstract string Name { get; }      // subclasses MUST override
    public abstract double Area();

    // virtual: subclasses MAY override
    public virtual string Describe() => $"{Name} with area {Area():F2}";
}
```

The Python version uses an abstract base class:

Python

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @property
    @abstractmethod
    def name(self) -> str: ...

    @abstractmethod
    def area(self) -> float: ...

    def describe(self) -> str:
        return f"{self.name} with area {self.area():.2f}"
```

9.2 The Subclasses

Tutorial/Ch09Interfaces.cs

```
public class Circle : Shape                    // Python: class Circle(Shape):
{
    public double Radius { get; }

    public Circle(double radius) => Radius = radius;

    public override string Name => "Circle";
    public override double Area() => Math.PI * Radius * Radius;
}

public class Rectangle : Shape
{
    public double Width { get; }
    public double Height { get; }

    public Rectangle(double width, double height)
    {
        Width = width;
        Height = height;
    }

    public override string Name => "Rectangle";
    public override double Area() => Width * Height;
}

// sealed: nothing may inherit from Square.
// : base(...) calls the parent constructor (Python: super().__init__(...))
public sealed class Square : Rectangle
{
    public Square(double side) : base(side, side) { }

    public override string Name => "Square";
    public override string Describe() => base.Describe() + " (all sides equal)";
}
```

Keyword	Meaning
`: Shape`	Inherit from `Shape`. Python: `class Circle(Shape):`
`abstract`	No body here; every subclass *must* override it. An abstract class cannot be created with `new`.
`virtual`	Has a body; subclasses *may* override it.
`override`	This method replaces the parent's abstract or virtual one. Required; C# never overrides silently.
`sealed`	No class may inherit from this one.

Keyword	Meaning
`base`	The parent class. `base.Describe()` calls the parent's version; `: base(side, side)` calls its constructor. Python: `super()`.

In Python every method can be overridden. In C# a method can be overridden only if an ancestor class declares it `virtual`, `abstract` or `override`. An `override` stays overridable further down: `Square` overrides `Name`, which `Rectangle` already overrode. Marking a method `sealed override` stops that.

9.3 Polymorphism

Polymorphism means one call runs different code depending on the object's actual class. `TotalArea` accepts any sequence of `IShape` objects and calls `Area()` on each. Each object runs its own class's `Area()`.

`Run` passes a `List<Shape>` where `IEnumerable<IShape>` is expected. This is allowed because every `Shape` is an `IShape`, and `IEnumerable` only reads items, so nothing of the wrong type can be added through it. The rule is called **covariance**.

Tutorial/Ch09Interfaces.cs

```
// Accepts any sequence of anything that implements IShape
public static double TotalArea(IEnumerable<IShape> shapes)
{
    double total = 0;
    foreach (var shape in shapes)
    {
        total += shape.Area();            // each object runs its own Area()
    }
    return total;
}

// in Run():
var shapes = new List<Shape>
{
    new Circle(1), new Rectangle(2, 3), new Square(2),
};

foreach (var shape in shapes)
{
    Console.WriteLine(shape.Describe());
}
Console.WriteLine($"Total area: {TotalArea(shapes):F2}");
```

Output

```
Circle with area 3.14
Rectangle with area 6.00
Square with area 4.00 (all sides equal)
Total area: 13.14
```

9.4 Type Tests and Pattern Matching

`shape is Circle c` tests the type and, if it matches, gives you a variable `c` of type `Circle`. It replaces Python's `isinstance` plus a separate variable.

Tutorial/Ch09Interfaces.cs

```
// in Run():
// Type test plus cast in one step (Python: isinstance)
foreach (var shape in shapes)
{
    if (shape is Circle c)
    {
        Console.WriteLine($"found a circle of radius {c.Radius}");
    }
}

// Pattern matching on type. First matching arm wins.
string kind = shapes[2] switch
{
    Square sq => $"square of side {sq.Width}",
    Rectangle r => $"rectangle {r.Width} x {r.Height}",
    _ => "something else",
};
Console.WriteLine(kind);
```

The cases of a switch expression, called arms, are tested in order. `Square` must come before `Rectangle`, because a square is also a rectangle and the first match wins.

9.5 Interface or Abstract Class?

This is a standard interview question. The short answer:

	Interface	Abstract class
How many per class	Any number	One
Stored data (fields)	No	Yes
Constructors	No	Yes
Method bodies	Optional default bodies since C# 8; usually none	Yes, mixed with abstract members

	Interface	Abstract class
Use when	Unrelated classes share a capability ("can be discounted")	Related classes share code ("is a shape")

10 LINQ

LINQ (Language Integrated Query) is a set of methods for filtering, transforming, sorting and grouping collections. It does the work of Python's list comprehensions, `sorted`, `sum`, `any` and `itertools`. Most LINQ methods take a lambda. Methods that return a sequence, such as `Where` and `Select`, can be chained left to right; methods such as `First`, `Sum` and `ToList` end the chain with a single value or a list. File: `src/Tutorial/Ch10Linq.cs`.

10.1 Python to LINQ

Python	LINQ
`[x for x in xs if x > 0]`	`xs.Where(x => x > 0)`
`[x * 2 for x in xs]`	`xs.Select(x => x * 2)`
`sorted(xs, key=k)`	`xs.OrderBy(x => k(x))`
`sorted(xs, key=k, reverse=True)`	`xs.OrderByDescending(...)`
`xs[0] / next(iter(xs))`	`xs.First()`
`next((x for x in xs if p(x)), None)`	`xs.FirstOrDefault(x => p(x))`
`any(p(x) for x in xs)`	`xs.Any(x => p(x))`
`all(p(x) for x in xs)`	`xs.All(x => p(x))`
`sum(x.price for x in xs)`	`xs.Sum(x => x.Price)`
`len([x for x in xs if p(x)])`	`xs.Count(x => p(x))`
`max(xs, key=k)`	`xs.MaxBy(x => k(x))`
`set(xs)`	`xs.Distinct()`
`xs[:3] / xs[3:]`	`xs.Take(3) / xs.Skip(3)`
`defaultdict(list) grouping loop`	`xs.GroupBy(x => x.Key)`
`list(...) / {k: v for ...}`	`.ToList() / .ToDictionary(...)`

10.2 Walkthrough

The examples query a list of `Employee` records.

Tutorial/Ch10Linq.cs

```
public record Employee(string Name, string Dept, decimal Salary, int Years);

// in Run():
// Python: [e.name for e in staff if e.salary > 100_000]
List<string> highEarners = Staff
    .Where(e => e.Salary > 100_000m)
    .Select(e => e.Name)
    .ToList();
Console.WriteLine($"High earners: {string.Join(", ", highEarners)}");

// Python: sorted(staff, key=lambda e: (-e.years, e.name))
var bySeniority = Staff
    .OrderByDescending(e => e.Years)
    .ThenBy(e => e.Name);
Console.WriteLine($"Most senior: {bySeniority.First().Name}");
```

Output

```
High earners: Ada, Grace, Barbara
Most senior: Grace
```

`e => e.Salary > 100_000m` is the lambda from section 7.4: for each employee `e`, keep it if the salary is over 100,000. `.ToList()` turns the result into a `List`.

Tutorial/Ch10Linq.cs

```
// in Run():
// GroupBy, then shape each group into a new anonymous object
var byDept = Staff
    .GroupBy(e => e.Dept)
    .Select(g => new
    {
        Dept = g.Key,
        Count = g.Count(),
        Avg = g.Average(e => e.Salary),
    })
    .OrderBy(x => x.Dept);
foreach (var d in byDept)
{
    Console.WriteLine($"{d.Dept,-12} {d.Count} people, avg {d.Avg:0}");
}
```

Output

```
Engineering  3 people, avg 117667
Finance      1 people, avg 105000
Sales        2 people, avg 89000
```

- `GroupBy` produces one group per department. Each group `g` has a `Key` (the department) and is itself a sequence of employees.

- `new { Dept = ..., Count = ... }` creates an **anonymous type**: an object with those properties and no declared class. It is like building a small dict, but typed.
- `{d.Dept,-12}` pads the text to 12 characters, left-aligned. Python: `{d.dept:<12}`.

10.3 Deferred Execution

Deferred execution means a LINQ query does not run when you write it; it runs each time you loop over it. It is lazy like a Python generator expression, with one difference: a generator is used up after one loop, while a LINQ query runs again on every loop.

Tutorial/Ch10Linq.cs

```
// in Run():
// Deferred execution: a query runs when you loop over it, and again
// on every later loop. (Lazy like a Python generator, but reusable.)
var numbers = new List<int> { 1, 2, 3 };
var evens = numbers.Where(n => n % 2 == 0);  // nothing runs yet
numbers.Add(4);
Console.WriteLine($"evens: {string.Join(",", evens)}");  // 2,4
```

The output is `2,4`: the 4 was added after the query was written, but before it ran. Two consequences matter in practice:

- Looping over the same query twice runs it twice. Call `.ToList()` to run it once and keep the result.
- `IEnumerable<T>`, the type a query returns, means "something you can loop over". It does not mean the items exist yet. A `List<T>` holds them in memory.

Entity Framework, the standard database library, uses `IQueryable<T>`. The same LINQ methods are translated into SQL and run in the database instead of in memory.

10.4 Query Syntax

LINQ has a second, SQL-like syntax. The compiler turns it into the same method calls. Method syntax is more common in modern code; read both.

Tutorial/Ch10Linq.cs

```
// in Run():
// Query syntax: the same thing written SQL-style
var engineers = from e in Staff
                where e.Dept == "Engineering"
                orderby e.Salary descending
                select e.Name;
```

10.5 Extension Methods

`Where` and `Select` are not defined on `List`. They are **extension methods**: static methods that the compiler lets you call as if they belonged to the type. The keyword `this` before the first parameter makes a method an extension method.

Tutorial/Ch10Linq.cs

```
// Extension method: a static method you call as if it belonged to the type.
// The "this" before the first parameter is what makes it one.
// LINQ's Where, Select, OrderBy are all extension methods on IEnumerable<T>.
public static class EmployeeExtensions
{
    public static decimal TotalSalary(this IEnumerable<Employee> staff)
        => staff.Sum(e => e.Salary);
}

// in Run():
// Calling the extension method defined above
decimal engPayroll = Staff.Where(e => e.Dept == "Engineering").TotalSalary();
Console.WriteLine($"Engineering payroll: {engPayroll}");
```

`staff.TotalSalary()` compiles to `EmployeeExtensions.TotalSalary(staff)`. Python has no equivalent; the nearest is a plain function.

11 Exceptions and Cleanup

C# exceptions work like Python's: `throw` raises one, and `try` / `catch` / `finally` handles it. File: `src/Tutorial/Ch11Exceptions.cs`.

11.1 Throwing and Catching

Tutorial/Ch11Exceptions.cs

```
public static int ParseQuantity(string text)
{
    if (!int.TryParse(text, out int qty))
    {
        // throw = Python's raise
        throw new FormatException($"'{text}' is not a whole number.");
    }
    if (qty < 0)
    {
        throw new ArgumentOutOfRangeException(
            nameof(text), "Quantity cannot be negative.");
    }
    return qty;
}

// in Run():
foreach (var input in new[] { "5", "abc", "-2" })
{
    try
    {
        Console.WriteLine($"parsed {ParseQuantity(input)}");
    }
    catch (FormatException ex)          // Python: except ValueError as ex
    {
        Console.WriteLine($"bad format: {ex.Message}");
    }
    catch (ArgumentOutOfRangeException ex)
    {
        Console.WriteLine($"out of range: parameter '{ex.ParamName}'");
    }
    finally                             // always runs
    {
        Console.WriteLine($"  (checked '{input}')");
    }
}
```

Output

```
parsed 5
  (checked '5')
bad format: 'abc' is not a whole number.
  (checked 'abc')
out of range: parameter 'text'
  (checked '-2')
```

Python	C#
`raise ValueError("...")`	`throw new ArgumentException("...");`
`except ValueError as ex:`	`catch (ArgumentException ex)`
`except Exception:`	`catch (Exception)`
`raise (re-raise)`	`throw;`
`str(ex)`	`ex.Message`
`KeyError / IndexError`	`KeyNotFoundException / IndexOutOfRangeException`
`AttributeError on None`	`NullReferenceException`

The most common exceptions are `ArgumentException` and `ArgumentNullException` (bad input to a method), `InvalidOperationException` (the object is in the wrong state for this call), `NullReferenceException`, and `FormatException` (text that does not parse).

> **Interview point · `throw;` versus `throw ex;`**
>
> Inside a `catch`, `throw;` re-throws the same exception with its original stack trace. `throw ex;` re-throws it but resets the stack trace to the current line, which hides where the error started. Use `throw;`.

11.2 Custom Exceptions

A custom exception is a class that inherits from `Exception`. Add properties for any extra data the catching code needs, such as `OrderId`. `: base(message)` passes the message to `Exception`'s constructor.

Tutorial/Orders/Models.cs

```
// Raised when an order cannot be completed.
public class OrderException : Exception
{
    public string OrderId { get; }

    public OrderException(string orderId, string message) : base(message)
    {
        OrderId = orderId;
    }
}
```

11.3 using and IDisposable

A `using` block guarantees that cleanup runs when the block ends, even if an exception is thrown. It is Python's `with` statement. The cleanup method is `Dispose()`, from the `IDisposable` interface, which plays the role of `__exit__`.

Tutorial/Ch11Exceptions.cs

```
// in Run():
// using = Python's with. Dispose() runs when the block ends, even on error.
using (var log = new TempLog("demo"))
{
    log.Write("inside the using block");
}

// IDisposable is the interface "using" relies on (Python: __enter__/__exit__)
public class TempLog : IDisposable
{
    private readonly string _name;

    public TempLog(string name)
    {
        _name = name;
        Console.WriteLine($"open {_name}");
    }

    public void Write(string message) => Console.WriteLine($"[{_name}]
{message}");

    public void Dispose() => Console.WriteLine($"close {_name}");
}
```

Output

```
open demo
[demo] inside the using block
close demo
```

`Dispose` releases a file handle or connection immediately; the garbage collector frees memory only at an unpredictable time. Files, database connections and network streams all implement `IDisposable`.

Modern code often writes the short form, which disposes at the end of the enclosing block:

C#

```
using var reader = new StreamReader("data.txt");   // Python: with open(...) as
reader:
string text = reader.ReadToEnd();
```

12 Async and Await

`async` and `await` let a method wait for slow input or output (a web request, a database query, a file) without blocking a thread. The keywords and their purpose match Python's `asyncio`. File: `src/Tutorial/Ch12Async.cs`.

12.1 Walkthrough

Tutorial/Ch12Async.cs

```
// async method returning Task<string> (Python: async def fetch(...) -> str)
public static async Task<string> FetchAsync(string name, int delayMs)
{
    await Task.Delay(delayMs);            // Python: await asyncio.sleep(...)
    return $"{name} finished after {delayMs} ms";
}

public static async Task RunAsync()
{
    var timer = Stopwatch.StartNew();

    // One after the other: about 600 ms in total
    string first = await FetchAsync("first", 300);
    string second = await FetchAsync("second", 300);
    long ms = timer.ElapsedMilliseconds / 100 * 100;   // round down to 100s
    Console.WriteLine($"sequential took ~{ms} ms");

    timer.Restart();

    // Both at once: about 300 ms in total.
    // Calling an async method STARTS it.
    // (A Python coroutine does nothing until awaited.)
    Task<string> a = FetchAsync("A", 300);
    Task<string> b = FetchAsync("B", 300);
    string[] results = await Task.WhenAll(a, b);   // Python: asyncio.gather
    ms = timer.ElapsedMilliseconds / 100 * 100;
    Console.WriteLine($"concurrent took ~{ms} ms");

    foreach (var line in new[] { first, second }.Concat(results))
    {
        Console.WriteLine(line);
    }
}
```

Output

```
sequential took ~600 ms
concurrent took ~300 ms
first finished after 300 ms
second finished after 300 ms
A finished after 300 ms
B finished after 300 ms
```

- `Task<string>` is an object that represents an operation in progress; when the operation finishes, the task holds its `string` result. `Task` without a type argument returns nothing. Python's closest equivalent is an `asyncio.Task` from `asyncio.create_task`, which also starts running at once.
- `await` pauses this method until the task finishes, and frees the thread for other work meanwhile.
- `Task.WhenAll(a, b)` waits for both. It is `asyncio.gather`.
- By convention, async method names end in `Async`.

12.2 Differences from Python

- **Calling an async method starts it.** `FetchAsync("A", 300)` begins running at once and returns a running `Task`. A Python coroutine does nothing until it is awaited or scheduled. This is why lines `a` and `b` above run at the same time.
- **No event loop to start.** There is no `asyncio.run()`. `Main` (or top-level code) can itself use `await`.
- **Real threads.** After an `await`, the method may continue on a different thread-pool thread. For CPU-heavy work, `Task.Run(() => ...)` runs code on another thread, and threads run in parallel.

12.3 Rules Interviewers Check

- Do not call `.Result` or `.Wait()` on a task. Both block the thread until the task ends, which wastes the thread and can deadlock in UI and older ASP.NET apps. Use `await`.
- Do not write `async void` methods, except for event handlers. A caller cannot await them, and their exceptions crash the process. Return `Task`.
- Callers of an async method usually become async too, so that they can `await` it. This is normal.

13 Walkthrough: The Order System

The order system in `src/Tutorial/Orders/` turns an order into a receipt. It checks stock, removes the items from stock, applies a discount and adds tax. It is a small, realistic program that uses records, classes, an interface, a dictionary, LINQ and exceptions together. This is the code to practise changing.

13.1 The Data Types

Tutorial/Orders/Models.cs

```
// One item for sale. A record: immutable, compared by value.
public record Product(string Sku, string Name, decimal Price);

// One row of an order: which product, how many.
public record OrderLine(Product Product, int Quantity)
{
    public decimal Subtotal => Product.Price * Quantity;
}

// An order is mutable (lines get added), so it is a class, not a record.
public class Order
{
    public string Id { get; }
    public string Customer { get; }
    public List<OrderLine> Lines { get; } = new();

    public Order(string id, string customer)
    {
        Id = id;
        Customer = customer;
    }

    public decimal Subtotal => Lines.Sum(line => line.Subtotal);

    public void Add(Product product, int quantity)
    {
        if (quantity <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(quantity), "Quantity must be positive.");
        }
        Lines.Add(new OrderLine(product, quantity));
    }
}

// What checkout returns.
public record Receipt(string OrderId, decimal Subtotal,
                      decimal Discount, decimal Tax, decimal Total);
```

- `Product`, `OrderLine` and `Receipt` are records: data that does not change after creation.
- `OrderLine` adds a computed property, `Subtotal`, inside the record's braces.
- `Order` is a class because lines are added to it over time.
- `Lines { get; } = new();` creates an empty list when the order is created. The list itself can grow, but the property can never be pointed at a different list.
- `Lines.Sum(line => line.Subtotal)` is LINQ: Python's `sum(l.subtotal for l in self.lines)`.

13.2 Discount Rules

`IDiscountPolicy` is an interface with one method. Each discount rule is a class that implements it. `OrderService` depends only on the interface, so a new rule needs a new class and no change to `OrderService`.

Tutorial/Orders/Discounts.cs

```
// Discount rules sit behind an interface, so OrderService works with any of them.
public interface IDiscountPolicy
{
    decimal DiscountFor(Order order);
}

public class NoDiscount : IDiscountPolicy
{
    public decimal DiscountFor(Order order) => 0m;
}

// A percentage off any order whose subtotal reaches a threshold.
public class BulkDiscount : IDiscountPolicy
{
    public decimal Threshold { get; }
    public decimal Rate { get; }

    public BulkDiscount(decimal threshold = 100m, decimal rate = 0.10m)
    {
        Threshold = threshold;
        Rate = rate;
    }

    public decimal DiscountFor(Order order)
    {
        if (order.Subtotal < Threshold)
        {
            return 0m;
        }
        return Math.Round(order.Subtotal * Rate, 2);
    }
}
```

13.3 Checkout

`Checkout` either returns a `Receipt` or throws `OrderException`. It checks every line before it removes any stock, so a failed order leaves the stock unchanged.

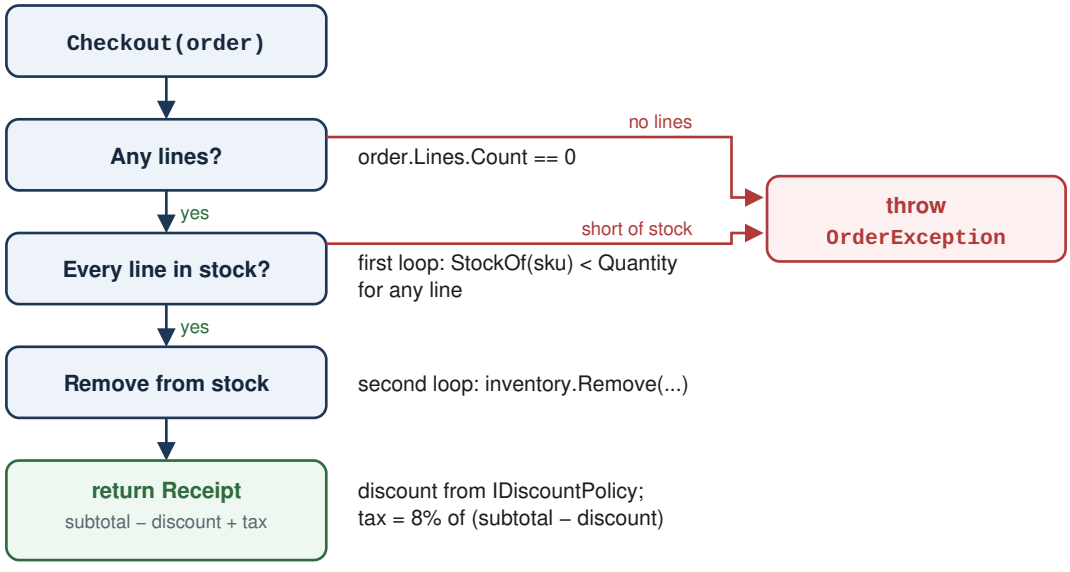


Figure 5. `OrderService.Checkout`. Either check can throw; only a fully valid order changes stock.

Tutorial/Orders/OrderService.cs

```
// Tracks how many of each product are on the shelf.
public class Inventory
{
    private readonly Dictionary<string, int> _stock = new();

    public void Restock(string sku, int quantity)
    {
        _stock[sku] = StockOf(sku) + quantity;
    }

    // GetValueOrDefault returns 0 when the sku is missing
    public int StockOf(string sku) => _stock.GetValueOrDefault(sku);

    public void Remove(string sku, int quantity)
    {
        _stock[sku] = StockOf(sku) - quantity;
    }
}
```

Tutorial/Orders/OrderService.cs

```
// Turns an Order into a Receipt.
public class OrderService
{
    public const decimal TaxRate = 0.08m;

    private readonly Inventory _inventory;
    private readonly IDiscountPolicy _discounts;

    // Dependency injection: the service receives its collaborators
    // instead of creating them. Tests can pass in fakes.
    public OrderService(Inventory inventory, IDiscountPolicy discounts)
    {
        _inventory = inventory;
        _discounts = discounts;
    }

    public Receipt Checkout(Order order)
    {
        if (order.Lines.Count == 0)
        {
            throw new OrderException(order.Id, "Order is empty.");
        }

        // Check every line first, so a failure leaves the stock untouched.
        foreach (var line in order.Lines)
        {
            int available = _inventory.StockOf(line.Product.Sku);
            if (available < line.Quantity)
            {
                throw new OrderException(order.Id, $"Only {available} of " +
                    $"{line.Product.Name} in stock, {line.Quantity} requested.");
            }
        }

        foreach (var line in order.Lines)
        {
            _inventory.Remove(line.Product.Sku, line.Quantity);
        }

        decimal subtotal = order.Subtotal;
        decimal discount = _discounts.DiscountFor(order);
        decimal tax = Math.Round((subtotal - discount) * TaxRate, 2);
        decimal total = subtotal - discount + tax;
        return new Receipt(order.Id, subtotal, discount, tax, total);
    }
}
```

13.4 Dependency Injection

Dependency injection (DI) means a class receives the objects it works with through its constructor instead of creating them. `OrderService` receives an `Inventory` and an `IDiscountPolicy`. It never writes `new BulkDiscount()` itself.

This gives two benefits:

- **Swapping behaviour.** The caller decides which discount rule applies. `Ch13Orders` passes `BulkDiscount`; a test passes `NoDiscount`.
- **Testing.** A test can pass a simplified or fake object. In larger code bases, a mocking library such as Moq generates fakes from an interface.

`readonly` on the two fields means they can be assigned only in the constructor. A `const` such as `TaxRate` is fixed at compile time and is automatically static.

13.5 Running It

Tutorial/Ch13Orders.cs

```
// in Run():
var keyboard = new Product("KB-01", "Keyboard", 45.00m);
var mouse = new Product("MS-02", "Mouse", 25.00m);
var monitor = new Product("MN-03", "Monitor", 180.00m);

var inventory = new Inventory();
inventory.Restock(keyboard.Sku, 10);
inventory.Restock(mouse.Sku, 10);
inventory.Restock(monitor.Sku, 1);

var service = new OrderService(inventory, new BulkDiscount());

var small = new Order("ORD-1", "Ada");
small.Add(mouse, 2);
PrintReceipt(service.Checkout(small));

var large = new Order("ORD-2", "Grace");
large.Add(keyboard, 1);
large.Add(monitor, 1);
PrintReceipt(service.Checkout(large));

var tooMany = new Order("ORD-3", "Linus");
tooMany.Add(monitor, 1);          // the only monitor was sold in ORD-2
try
{
    service.Checkout(tooMany);
}
catch (OrderException ex)
{
    Console.WriteLine($"{ex.OrderId} rejected: {ex.Message}");
}

Console.WriteLine($"Mice left: {inventory.StockOf(mouse.Sku)}");
```

Run it with `dotnet run --project src/Tutorial -- 13`:

Output

```
ORD-1: subtotal 50.00 - discount 0.00 + tax 4.00 = 54.00
ORD-2: subtotal 225.00 - discount 22.50 + tax 16.20 = 218.70
ORD-3 rejected: Only 0 of Monitor in stock, 1 requested.
Mice left: 8
```

ORD-1 is below the 100.00 threshold, so it gets no discount. ORD-2 is 225.00, so it gets 10% off (22.50), and tax is 8% of the remaining 202.50. ORD-3 fails because ORD-2 bought the only monitor.

13.6 The Unit Tests

The tests use **xUnit**, the most common .NET test framework. NUnit and MSTest are the other two you may hear named. Each test method follows **Arrange, Act, Assert**: set up objects, call the method under test, check the result.

tests/Tutorial.Tests/OrderServiceTests.cs

```
// Helper: a service with 5 of each product and the given discount rule
private static (OrderService Service, Inventory Stock) Build(
    IDiscountPolicy discounts)
{
    var stock = new Inventory();
    stock.Restock(Mouse.Sku, 5);
    stock.Restock(Monitor.Sku, 5);
    return (new OrderService(stock, discounts), stock);
}

[Fact]
public void Checkout_NotEnoughStock_ThrowsAndLeavesStockAlone()
{
    var (service, stock) = Build(new NoDiscount());
    var order = new Order("T-3", "Linus");
    order.Add(Mouse, 1);
    order.Add(Monitor, 99);

    var ex = Assert.Throws<OrderException>(() => service.Checkout(order));

    Assert.Equal("T-3", ex.OrderId);
    Assert.Equal(5, stock.StockOf(Mouse.Sku));   // the mouse was not removed
}
```

pytest	xUnit
`def test_x():`	`[Fact] public void X()`
`@pytest.mark.parametrize`	`[Theory] + [InlineData(...)]`
`assert actual == expected`	`Assert.Equal(expected, actual)`

pytest	xUnit
`with pytest.raises(E):`	`Assert.Throws<E>(() => ...)`
`pytest`	`dotnet test`

Text in square brackets, such as `[Fact]`, is an **attribute**: a label attached to a class or method that other code reads at run time. The test runner finds tests by looking for `[Fact]`. Attributes look like Python decorators but only attach information; they do not wrap the method.

Note the argument order: `Assert.Equal` takes the expected value first.

14 Walkthrough: The Web API

Most C# jobs build web back ends with **ASP.NET Core**, the .NET web framework. Its "minimal API" style resembles FastAPI or Flask: you map a URL to a function. The project `src/Tutorial.Api` serves the order system over HTTP in about 60 lines.

14.1 Registering Services

Tutorial.Api/Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// 1. Build the shared objects and register them with the
//    dependency-injection (DI) container.
var catalog = new Dictionary<string, Product>
{
    ["KB-01"] = new Product("KB-01", "Keyboard", 45.00m),
    ["MS-02"] = new Product("MS-02", "Mouse", 25.00m),
    ["MN-03"] = new Product("MN-03", "Monitor", 180.00m),
};
var inventory = new Inventory();
foreach (var sku in catalog.Keys)
{
    inventory.Restock(sku, 5);
}

builder.Services.AddSingleton(catalog);
builder.Services.AddSingleton(inventory);
builder.Services.AddSingleton<IDiscountPolicy>(new BulkDiscount(100m, 0.10m));
builder.Services.AddSingleton<OrderService>();   // DI calls its constructor

var app = builder.Build();
```

`builder.Services` is the list of registrations for the **DI container**, the object that creates registered types and hands them to code that asks for them. `builder.Build()` turns the list into the container. `AddSingleton<OrderService>()` registers the class without creating it. The first time something needs an `OrderService`, the container reads its constructor, finds an `Inventory` and an `IDiscountPolicy` already registered, and passes them in. This is section 13.4's dependency injection, done automatically.

Every registration has a **lifetime**. Interviewers ask about these three:

Lifetime	The container creates	Typical use
`AddSingleton`	One object for the whole application	Caches, configuration, this demo's shared stock

Lifetime	The container creates	Typical use
`AddScoped`	One object per HTTP request	Database context (EF Core `DbContext`)
`AddTransient`	A new object every time one is requested	Small stateless helpers

14.2 Endpoints

Tutorial.Api/Program.cs

```
// 2. Map URLs to handler lambdas. ASP.NET fills each parameter from the
//    URL, the JSON request body, or the DI container.
app.MapGet("/products", (Dictionary<string, Product> catalog) => catalog.Values);

app.MapGet("/stock/{sku}", (string sku, Inventory inventory) =>
    Results.Ok(new { sku, inStock = inventory.StockOf(sku) }));

app.MapPost("/orders", (OrderRequest request,
                        Dictionary<string, Product> catalog,
                        OrderService service) =>
{
    var order = new Order($"ORD-{Random.Shared.Next(1000, 9999)}",
request.Customer);
    try
    {
        foreach (var line in request.Lines)
        {
            if (!catalog.TryGetValue(line.Sku, out Product? product))
            {
                return Results.NotFound(new { error = $"Unknown SKU
{line.Sku}" });
            }
            order.Add(product, line.Quantity);
        }
        return Results.Ok(service.Checkout(order));   // record -> JSON
    }
    catch (Exception ex) when (ex is OrderException or ArgumentException)
    {
        return Results.BadRequest(new { error = ex.Message });
    }
});

// The JSON body of POST /orders. JSON {"customer": ...} maps to Customer.
public record LineRequest(string Sku, int Quantity);
public record OrderRequest(string Customer, List<LineRequest> Lines);
```

- `MapGet` and `MapPost` connect an HTTP method and URL to a lambda. FastAPI writes the same thing as `@app.get("/products")`.
- ASP.NET fills each lambda parameter from one of three places. `{sku}` in the route fills `string sku`. A registered type, such as `Inventory`, comes from the DI

container. Any other class or record, such as `OrderRequest`, is read from the JSON request body.

- Returned objects are converted to JSON. Property names become camelCase: `OrderId` becomes `"orderId"`.
- `Results.Ok`, `Results.BadRequest` and `Results.NotFound` set the status code: 200, 400 and 404.
- `catch (Exception ex) when (...)` is an **exception filter**: the `catch` runs only if the condition is true.

14.3 Trying It

Terminal 1

```
dotnet run --project src/Tutorial.Api --urls http://localhost:5239
```

Terminal 2

```
curl localhost:5239/products
[{"sku":"KB-01","name":"Keyboard","price":45.00}, ...]

curl -X POST localhost:5239/orders -H 'Content-Type: application/json' \
     -d '{"customer":"Ada","lines":[{"sku":"MN-03","quantity":1}]}'
{"orderId":"ORD-1518","subtotal":180.00,"discount":18.00,"tax":12.96,"total":
174.96}

curl -X POST localhost:5239/orders -H 'Content-Type: application/json' \
     -d '{"customer":"Ada","lines":[{"sku":"MN-03","quantity":9}]}'
{"error":"Only 4 of Monitor in stock, 9 requested."}         (HTTP 400)
```

The order id is random, so yours will differ. Stop the server with Ctrl+C.

Larger applications use **controllers** instead of lambdas: classes whose methods are marked with attributes such as `[HttpGet("products")]`. The ideas are the same. Data storage usually goes through **Entity Framework Core** (EF Core), an ORM comparable to SQLAlchemy or the Django ORM.

15 Hands-on Drills

Each drill is a small change to the companion project, ordered from easiest to hardest. Run the stated command after each change. Solutions are in appendix A. Before starting, check that the project is clean: `dotnet test` should report 8 passed.

> **Undo.** The project is a git repository. After a drill, `git checkout -- .` restores every file to its committed state. To keep your answers instead, work on a branch: `git switch -c drills`.

15.1 Drill 1: Read a Failing Test

In `Orders/OrderService.cs`, change `TaxRate` from `0.08m` to `0.10m`.

- Predict the new totals for ORD-1 and ORD-2, then check with `dotnet run --project src/Tutorial -- 13`.
- Run `dotnet test`. Two tests fail. Find the "Expected" and "Actual" lines in the output and explain each number.
- Change the rate back.

15.2 Drill 2: Read Compiler Errors

Make each change to `Ch03Basics.cs` on its own, run `dotnet build`, read the error, and undo it. The error format is `File(line,column): error CODE: message`.

Change	Error you will see
Delete the `;` after `int year = 1843`	`CS1002: ; expected`
Change `1843` to `"1843"`	`CS0029: Cannot implicitly convert type 'string' to 'int'`
Change `Console` to `console`	`CS0103: The name 'console' does not exist in the current context`
Add the line `var acct = new BankAccount();`	`CS7036: There is no argument given that corresponds to the required parameter 'owner'`

15.3 Drill 3: Find a Bug with a Test

`BankAccount.Withdraw` accepts a negative amount, and `Withdraw(-50m)` adds money. `Deposit` rejects non-positive amounts, but `Withdraw` does not.

1. In `BankAccountTests.cs`, add a `[Fact]` named `Withdraw_Negative_Throws` that expects `ArgumentOutOfRangeException`.
2. Run `dotnet test` and watch the new test fail.
3. Fix `Withdraw` and watch it pass. This order (failing test, then fix) is test-driven development.

15.4 Drill 4: Add a Method

Add `public void TransferTo(BankAccount other, decimal amount)` to `BankAccount`. It moves money from this account to `other`. Add a test that transfers 40 from an account holding 100 and checks both balances. Hint: reuse `Withdraw` and `Deposit`.

15.5 Drill 5: A New Subclass

In `Ch09Interfaces.cs`, add a `Triangle` class that inherits from `Shape`, with a base length and a height. Area is half of base times height. Add `new Triangle(3, 4)` to the `shapes` list. Expected new output lines: `Triangle with area 6.00` and `Total area: 19.14`.

Note: `base` is a keyword, so a parameter cannot be named `base`. Use `baseLength`.

15.6 Drill 6: A LINQ Query

In `Ch10Linq.Run`, print the highest-paid employee in each department. Expected output:

Output

```
Top in Engineering: Grace
Top in Sales: Alan
Top in Finance: Barbara
```

Hint: `GroupBy`, then for each group sort by salary, highest first, and take the first.

15.7 Drill 7: A New Discount Rule

In `Orders/Discounts.cs`, write `LoyaltyDiscount`, an `IDiscountPolicy` that gives 5% off to customers whose names are in a set passed to its constructor. In

`Ch13Orders.cs`, pass `new LoyaltyDiscount(new[] { "Ada" })` instead of `new BulkDiscount()`. Do not change `OrderService`. Expected:

Output

```
ORD-1: subtotal 50.00 - discount 2.50 + tax 3.80 = 51.30
ORD-2: subtotal 225.00 - discount 0.00 + tax 18.00 = 243.00
```

Afterwards, say in one sentence why `OrderService` did not need to change. That sentence is an interview answer about interfaces and dependency injection.

15.8 Drill 8: A New Endpoint

In `src/Tutorial.Api/Program.cs`, add `POST /stock/{sku}/{quantity}`, which restocks a product and returns the new stock level. Place it before `app.Run()`. Restart the server, then test it (a fresh server holds 5 monitors):

Output

```
curl -X POST localhost:5239/stock/MN-03/10
{"sku":"MN-03","inStock":15}
```

16 Interview Quick Reference

Each answer below is short enough to say aloud. Each one points to the example in the companion project that you can describe.

16.1 Likely Questions

What is the difference between a value type and a reference type?

A value type variable holds its data, and assignment copies it: numbers, `bool`, `struct`. A reference type variable holds a reference to an object, and assignment copies the reference, so two variables can share one object: classes, records, strings, arrays, lists. (Section 4.4.)

Class, struct or record?

A class is a reference type compared by identity, for objects with behaviour and state. A record is a reference type compared by value, for data. A struct is a value type, for small values. In the project, `BankAccount` is a class, `Receipt` a record, `Temperature` a struct.

Interface or abstract class?

A class can implement many interfaces but inherit only one class. An interface lists members that implementing classes must provide, and holds no stored data. An abstract class shares code and fields among related classes. I would use `IDiscountPolicy` as the interface example and `Shape` as the abstract class.

What do abstract, virtual and override mean?

`abstract`: no body here, subclasses must provide one. `virtual`: has a body, subclasses may replace it. `override`: this subclass method replaces the parent's. A method without `virtual` or `abstract` cannot be overridden.

What are the four pillars of object-oriented programming?

Encapsulation: a class guards its own data (`BankAccount._balance` is private; changes go through `Deposit`). **Abstraction**: callers call one public method and do not see the steps inside it (`Checkout`). **Inheritance**: a class reuses a parent's code (`Square : Rectangle`). **Polymorphism**: one call runs different code per type (`shape.Area()`).

What is SOLID?

Five design rules. **S**ingle responsibility: one reason to change per class. **O**pen/closed: add behaviour with new code, not edits (a new `IDiscountPolicy`). **L**iskov substitution: a subclass must work wherever its parent is expected. **I**nterface segregation: small, focused interfaces. **D**ependency inversion: depend on interfaces, not concrete classes (`OrderService` takes `IDiscountPolicy`).

What is dependency injection, and what are the lifetimes?

A class receives the objects it uses through its constructor instead of creating them, which makes them swappable and testable. ASP.NET Core's container creates them for you. Singleton: one for the app. Scoped: one per request. Transient: new each time. (Sections 13.4 and 14.1.)

What is LINQ, and what is deferred execution?

LINQ is a set of methods, such as `Where`, `Select` and `GroupBy`, for querying any collection. A query runs when you loop over it, not when you write it. `ToList()` runs it once and stores the result. Against a database, EF Core translates the same query into SQL. (Chapter 10.)

IEnumerable versus List?

`IEnumerable<T>` is an interface meaning "can be looped over"; it may be a lazy query. `List<T>` is a concrete in-memory collection with indexing and `Add`. Accept `IEnumerable` in parameters when you only loop, as `TotalArea` does.

How do async and await work?

An async method returns a `Task`. `await` pauses the method until the task finishes, releasing the thread meanwhile, so a server can handle other requests while waiting on I/O. Use Start several tasks, then `await Task.WhenAll(...)` to wait for all of them. Avoid `.Result`, `.Wait()` and `async void`. (Chapter 12.)

How is memory managed? What is IDisposable?

A garbage collector frees objects that nothing references. It runs at times you do not control. `IDisposable` and `using` release resources such as files and database connections at a known point, the way Python's `with` does. (Section 11.3.)

const, readonly and static?

`const`: a value fixed at compile time, such as `TaxRate`. `readonly`: a field assigned once, in the constructor, such as `OrderService._inventory`. `static`: belongs to the class, not to each object.

== versus Equals?

For classes, `==` compares identity unless the class overrides it. Records and `string` override it to compare contents, so `"a" == "a"` is true. `Equals` is the method form, which types override to define value equality.

Why is string immutable, and what is StringBuilder?

A `string` never changes; every edit makes a new string, as in Python. Immutability makes strings safe to share between threads and to use as dictionary keys. Building a long string with `+=` in a loop copies it every time. `StringBuilder` appends into one buffer, like collecting parts in a list and calling `"".join(parts)`.

What are generics?

Types and methods with a type placeholder, such as `List<T>`. One definition works for many types and stays type-checked. `where T : IShape` restricts what `T` can be.

What are delegates and events?

A delegate is a type for a function reference; `Func` and `Action` are the built-in ones, and a lambda can be assigned to one. An `event` is a delegate field that outside code can only subscribe to with `+=` or unsubscribe from with `-=`; the owning class calls the subscribers when something happens. Events are common in UI code. Python's closest match is passing callback functions.

How would you test this class?

With xUnit: one `[Fact]` per behaviour, each following arrange, act, assert. Pass fake dependencies through the constructor; a library such as Moq creates them from interfaces. `OrderServiceTests` is the example.

16.2 Talking About Your Background

> **Suggested answer**
>
> Say that your strongest language is Python, and that you have studied C# and can read it and make changes. Name the mappings you know: dataclasses to records, `with` to `using`, comprehensions to LINQ, `asyncio` to `Task`, pytest to xUnit; dependency injection works the same in both. Then prove it by explaining a piece of code. Interviewers trust a precise explanation of a small thing more than a broad claim.

16.3 Reading Unfamiliar C# Code

1. Open the `.csproj`: target framework (`net48` means .NET Framework, `net8.0` or later means modern .NET) and package references.
2. Find the entry point: `Program.cs`, either top-level statements or `static void Main`.
3. In a web app, read the `builder.Services.Add...` lines. They list every service registered with the DI container. Then find the routes: `Map...` calls or controller classes with `[Http...]` attributes.
4. For any class, read the constructor parameters first. They are its dependencies.
5. Names starting with `I` are interfaces. In an IDE, "Go to Implementation" finds the classes.
6. Read `using` lines at the top to see what each file depends on.
7. Methods ending in `Async` return a `Task`; follow the `await`s.
8. Read the tests. Test names state the intended behaviour in plain words.

16.4 Glossary

Term	Meaning
Assembly	A compiled `.dll` or `.exe`; the output of one project.
Attribute	A label in square brackets attached to code, read by tools or frameworks: `[Fact]`.
CLR	The runtime that executes .NET programs and collects garbage.
Delegate	A type describing a function signature; `Func`, `Action`.
DTO	Data transfer object: a type that only holds data, often a record.
EF Core	Entity Framework Core, the standard ORM.
IL	Intermediate Language: the compiled form inside a `.dll`.
JIT	Just-in-time compiler: turns each method's IL into machine code when the method is first called.
LINQ	Query methods over collections: `Where`, `Select`, ...
Namespace	A named group of types: `Tutorial.Orders`.
NuGet	The package registry and package format.
Property	A member read and written like a field, backed by get/set code.

Term	Meaning
Solution	A `.sln` or `.slnx` file listing projects that build together.
Task	An object representing an operation that finishes later; what async methods return.

A Drill Solutions

Each solution below was applied to a copy of the project and built. With all of them in place, all 10 tests pass and the outputs match those stated in chapter 15.

A.1 Drill 1

ORD-1: tax 5.00, total 55.00. ORD-2: tax 20.25, total 222.75. `dotnet test` reports:

Output

```
Failed Tutorial.Tests.OrderServiceTests.Checkout_LargeOrder_GetsBulkDiscount
  Assert.Equal() Failure: Values differ
  Expected: 174.96
  Actual:   178.20
Failed Tutorial.Tests.OrderServiceTests.Checkout_SmallOrder_AddsTaxOnly
  Assert.Equal() Failure: Values differ
  Expected: 4.00
  Actual:   5.00
```

178.20 is (180 − 18) × 1.10. The tests expect the 8% results, so they fail when the rate changes.

A.2 Drills 3 and 4

Tests/BankAccountTests.cs

```
[Fact]
public void Withdraw_Negative_Throws()
{
    var account = new BankAccount("Ada", 100m);
    Assert.Throws<ArgumentOutOfRangeException>(() => account.Withdraw(-50m));
}

[Fact]
public void TransferTo_MovesMoney()
{
    var source = new BankAccount("Ada", 100m);
    var target = new BankAccount("Grace");

    source.TransferTo(target, 40m);

    Assert.Equal(60m, source.Balance);
    Assert.Equal(40m, target.Balance);
}
```

Tutorial/Ch08Classes.cs

```
public void Withdraw(decimal amount)
{
    if (amount <= 0)
    {
        throw new ArgumentOutOfRangeException(
            nameof(amount), "Withdrawal must be positive.");
    }
    if (amount > _balance)
    {
        // ... unchanged ...
    }
    _balance -= amount;
}

public void TransferTo(BankAccount other, decimal amount)
{
    Withdraw(amount);          // throws before any money moves
    other.Deposit(amount);
}
```

Withdrawing first matters: if the balance is too low, `Withdraw` throws and `other` is never credited.

A.3 Drill 5

Tutorial/Ch09Interfaces.cs

```
public class Triangle : Shape
{
    public double BaseLength { get; }
    public double Height { get; }

    public Triangle(double baseLength, double height)
    {
        BaseLength = baseLength;
        Height = height;
    }

    public override string Name => "Triangle";
    public override double Area() => 0.5 * BaseLength * Height;
}

// in Run():
var shapes = new List<Shape>
{
    new Circle(1), new Rectangle(2, 3), new Square(2), new Triangle(3, 4),
};
```

A.4 Drill 6

Tutorial/Ch10Linq.cs

```
var topByDept = Staff
    .GroupBy(e => e.Dept)
    .Select(g => g.OrderByDescending(e => e.Salary).First());
foreach (var top in topByDept)
{
    Console.WriteLine($"Top in {top.Dept}: {top.Name}");
}
```

`g.MaxBy(e => e.Salary)` also works. It is typed as possibly null, so the compiler adds a nullable warning unless you write `g.MaxBy(e => e.Salary)!`.

A.5 Drill 7

Tutorial/Orders/Discounts.cs

```
// 5% off for customers on the loyalty list.
public class LoyaltyDiscount : IDiscountPolicy
{
    private readonly HashSet<string> _members;

    public LoyaltyDiscount(IEnumerable<string> members)
    {
        _members = new HashSet<string>(members);
    }

    public decimal DiscountFor(Order order)
    {
        if (!_members.Contains(order.Customer))
        {
            return 0m;
        }
        return Math.Round(order.Subtotal * 0.05m, 2);
    }
}
```

The one-sentence answer: `OrderService` depends on the `IDiscountPolicy` interface, not on any particular rule, so any class implementing that interface can be passed in without changing it.

A.6 Drill 8

Tutorial.Api/Program.cs

```
app.MapPost("/stock/{sku}/{quantity:int}",
    (string sku, int quantity, Inventory inventory) =>
{
    inventory.Restock(sku, quantity);
    return Results.Ok(new { sku, inStock = inventory.StockOf(sku) });
});
```

`{quantity:int}` is a route constraint: the URL matches only if that part is a whole number, and ASP.NET converts it to `int`.